no more filters in the pipeline, the endpoint handler is invoked. After the handler has executed, the filters may run further code.

Each endpoint filter has two parameters: a `context` parameter, which provides details about the selected endpoint handler, and the `next` parameter, which represents the filter pipeline. When you invoke the methodlike `next` parameter by calling `next(context)`, you invoke the remainder of the filter pipeline. If there are no more filters in the pipeline, you invoke the endpoint handler, as shown in figure 5.8.

Listing 5.9 shows how to run the same validation logic you saw in listing 5.8 in an endpoint filter. The filter function accesses the endpoint method arguments by using the `context.GetArgument<T>()` function, passing in a position; `0` is the first argument of your endpoint handler, `1` is the second argument, and so on. If the argument isn't valid, the filter function returns an `IResult` object response. If the argument is valid, the filter calls `await next(context)` instead, executing the endpoint handler.

Listing 5.9 Using `AddEndpointFilter` to extract common code

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();
var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit/{id}", (string id) =>
    _fruit.TryGetValue(id, out var fruit)
        ? TypedResults.Ok(fruit)
        : Results.Problem(statusCode: 404))
    .AddEndpointFilter(ValidationHelper.ValidateId);          ❶

app.Run();

class ValidationHelper
{
    internal static async ValueTask<object?> ValidateId(     ❷
        EndpointFilterInvocationContext context,              ❸
```